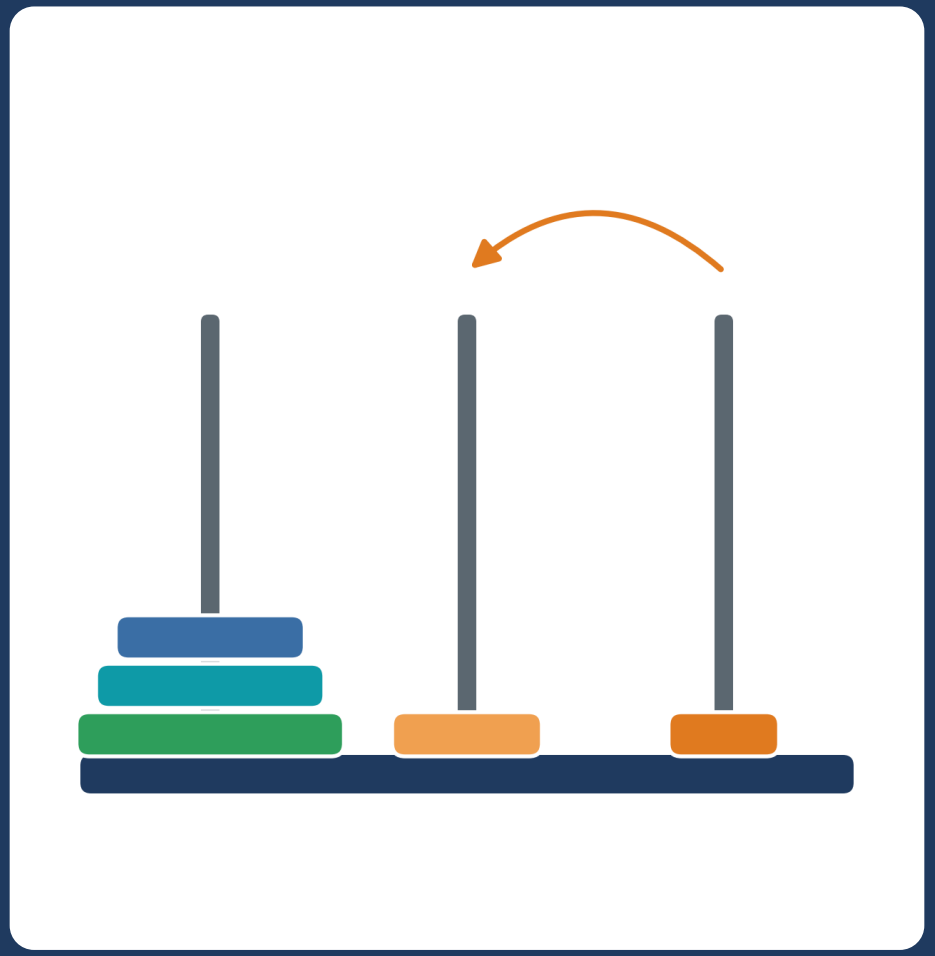
00 01 02 03 04 05 06 07 **08** 09 0A 0B 0C 0D

ALGORİTMA VE PROGRAMLAMA – I

HAFTA 09

Fonksiyonlar II: Kapsam, Ömür, Çağrı Yığını ve Özyinelemeye Giriş



Kapsam ve ömür · çok dosyalı derleme ve makrolar · çağrı yığını ve bellek düzeni · özyineleme, Fibonacci ve Hanoi Kuleleri

MODÜL IV · MODÜLER PROGRAMLAMA

Dr. Öğr. Üyesi Mehmet Sevri · RTEÜ Bilgisayar Mühendisliği · 2026–2027 Güz

14 HAFTALIK YOLCULUK

Yol haritasında neredeyiz?

H1 Bilgisayar ve Program	**H2** Algoritmik Düşünme	**H3** Değişkenler ve Veri Tipleri	**H4** Operatörler ve İfadeler	**H5** Karar Yapıları	**H6** Döngüler I	**H7** Döngüler II + Ara Proje
0x00	0x01	0x02	0x03	0x04	0x05	0x06

H8 Fonksiyonlar I	**H9** **Kapsam, Yığın, Özyineleme**	**H10** Diziler I	**H11** Diziler II: Matrisler	**H12** İşaretçiler ve Bellek	**H13** Karakter Dizileri	**H14** Yapılar, Dosyalar, Final
0x07	0x08	0x09	0x0A	0x0B	0x0C	0x0D

Vize: H1–H7 · Ara proje: H7 · Final projesi: H14

GERİ ÇAĞIRMA · H8 · PROTOTİP

kup'un tanımı main'den sonra, prototipi önce. Program ne yazdırır?

quiz/H09_Q01_prototip.c · satır 5–18

```c
#include <stdio.h>

int kup(int x);                     /* prototip (bildirim) */

int main(void)
{
    printf("kup(3) = %d\n", kup(3));
    return 0;
}

int kup(int x)                      /* tanım main'den SONRA */
{
    return x * x * x;
}
```

A `kup(3) = 9`

B `kup(3) = 27`

C Derleme hatası: kup bildirilmemiş

D Bağlama hatası: kup bulunamadı

GERİ ÇAĞIRMA · H8

Geçen haftadan · S2 (değerle aktarım) ve S3 (assert)

S2. `takas(x, y)` çağrısından sonra `x` ve `y` kaç olur?

quiz/H09_Q02_takas.c · satır 7–20

```
 7  void takas(int a, int b)
 8  {
 9      int gecici = a;
10      a = b;
11      b = gecici;
12  }
13
14  int main(void)
15  {
16      int x = 1, y = 2;
17      takas(x, y);
18      printf("x = %d, y = %d\n", x, y);
19      return 0;
20  }
```

S3. `mutlak(x)` doğru: |x| döndürür. Ne yazılır, çıkış durumu kaçtır?

quiz/H09_Q03_assert.c · satır 16–23

```
16  int main(void)
17  {
18      assert(mutlak(-5) == 5);
19      assert(mutlak(0) == 0);
20      assert(mutlak(7) == -7);
21      printf("Tüm testler geçti.\n");
22      return 0;
23  }
```

Yanıtlar: gerçek çıktılar

S1 · B Prototip imzayı önceden bildirir; prototipsiz GCC 13: **implicit declaration of function ‘kup’**

GERÇEK ÇIKTI

```
$ ./q01
kup(3) = 27
```

S2 · D `takas` kendi **çerçevesindeki kopyaları** değiştirir; çerçeve dönüşte yok olur (çözüm H12)

GERÇEK ÇIKTI

```
$ ./q02
x = 1, y = 2
```

S3 · C Üçüncü `assert` yanlış → `abort()`: çıkış durumu **134** = 128 + 6 (SIGABRT)

GERÇEK ÇIKTI

```
$ ./q03; echo "çıkış durumu: $?"
q03: H09_Q03_assert.c:20: main: Assertion `mutlak(7) == -7' failed.
bash: line 1:  9603 Aborted                 ./q03
çıkış durumu: 134
```

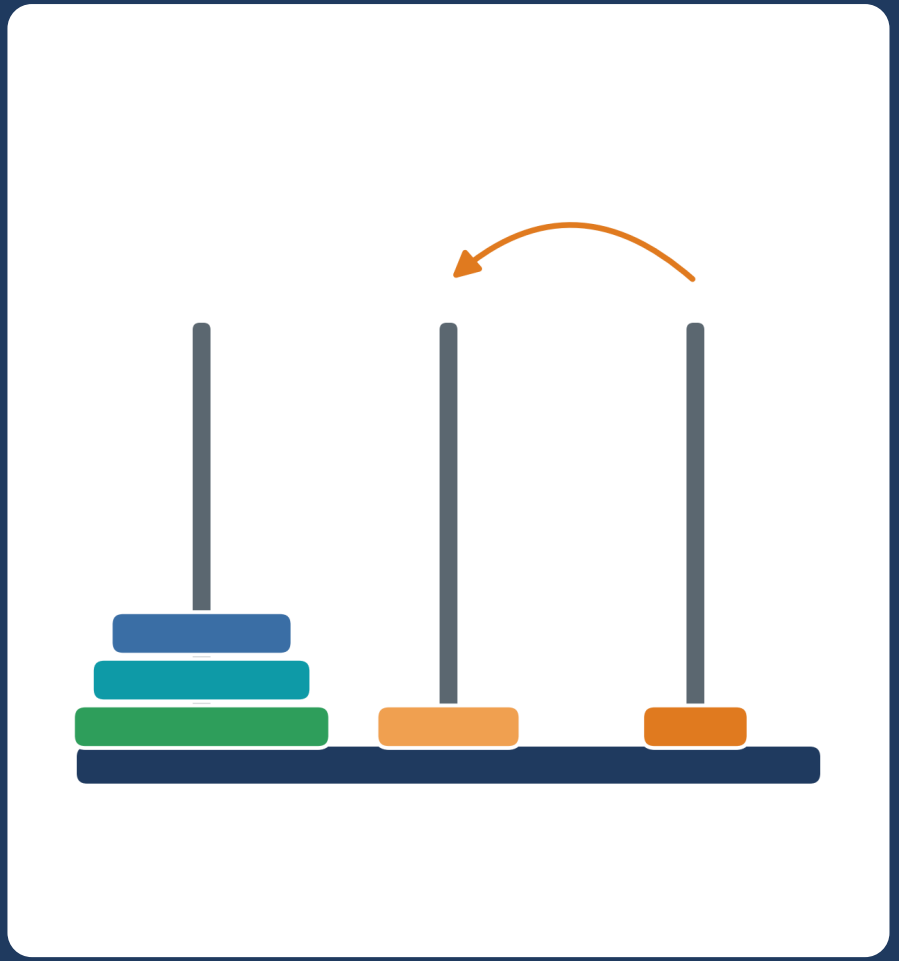
Bu hafta: **çerçeve**, **dönüş adresi** ve **özyineleme**

GERÇEK DÜNYADAN

1883

Hanoi Kuleleri ve 64 altın disk

- **Édouard Lucas**, 1883: takma ad **N. Claus de Siam**
- Kural: her hamlede en üstteki tek disk; büyük disk küçüğün üstüne konamaz
- Bulmacayla bir **efsane anlatılır**: 64 disk taşınınca dünyanın sonu gelecek
- 64 disk: $\mathbf{2^{64} - 1}$ = 18 446 744 073 709 551 615 hamle
- Saniyede bir hamle: ≈ $\mathbf{5{,}85 \times 10^{11}}$ **yıl** ≈ evrenin yaşının **42,4 katı**



Hinz vd. (2018) · Merrotsy (2015) · Planck Collaboration (2020) · süre hesabı: H09_K10_hanoi_64.c

BÜYÜK SORU

Haftanın Büyük Sorusu

Bir fonksiyon kendini çağırırsa ne olur ve bilgisayar bu çağrıların hangisinden nereye döneceğini nasıl hatırlar?



Her çağrının kendi n'si mi olur?

faktoriyel(4) çalışırken bellekte kaç tane n vardır?



Dönüş adresi nerede saklanır?

Binlerce iç içe çağrıda dönüşler neden karışmaz?



Bu düzenin bir sınırı var mı?

Kaç çağrı derinliğinde ne olur? Ölçeceğiz.

*Haftanın sonunda üç soruyu da **gerçek ölçümlerle** yanıtlayacağız.*

ÖĞRENME ÇIKTILARI

Bu hafta sonunda neler yapabileceksiniz?

ÖÇ 9.1	Blok, fonksiyon ve dosya kapsamını; otomatik ve statik ömrü ayırt eder.
ÖÇ 9.2	Çağrı yığınının çalışmasını yığın çerçeveleri üzerinden çizerek açıklar.
ÖÇ 9.3	Özyinelemeli bir fonksiyonun temel durumunu ve özyinelemeli adımını tanımlar; özyineleme ağacı ile yürütmeyi izler.
ÖÇ 9.4	Basit problemleri hem yinelemeli hem özyinelemeli çözer; iki yaklaşımı bellek ve zaman açısından karşılaştırır.
ÖÇ 9.5	Başlık (.h) ve kaynak (.c) dosyalarından oluşan çok dosyalı bir C projesi oluşturur ve derler.

01

Kapsam, ömür ve çok dosyalı programlar

Bir ad nerede görünür, bir değişken ne kadar yaşar, bir program dosyalara nasıl bölünür?

Kapsam: ad en içteki bloktan dışarı doğru aranır

DOSYA KAPSAMI · H09_01_kapsam_golgeleme.c

```
int x = 1;   /* global */
```

her yerden görünür; program boyunca yaşar

goster() — fonksiyon kapsamı

`printf(x)  →  1` *kendi x'i yok: dosya kapsamındaki x'i görür*

main() — fonksiyon gövdesi bloğu

`printf(x)  →  1` *henüz yerel x yok: global*

```
int x = 2;
```

globali gölgeler: bundan sonra x = 2

iç blok { ... }

```
int x = 3;
x = x * 10;  → 30
```

for bloğu

```
int i  (for başlığında)
int y = 10 * (i + 1);
```

`printf(x)  →  2` *iç x'in kapsamı ve ömrü bitti*

i, y ve iç x burada YOK → kullanılırsa 'undeclared' derleme hatası (K01)

GERÇEK ÇIKTI

renk = değeri yazılan x'in kapsamı

```
main başı      : x = 1 (global)
main           : x = 2
iç blok        : x = 3
iç blok sonu   : x = 30
bloktan sonra  : x = 2
for turu 0     : y = 10
for turu 1     : y = 20
goster()       : x = 1 (global)
```

Derleyici ne der? -Wshadow ve kapsam dışı kullanım

GÖLGELEME: YASAL AMA -WSHADOW UYARIR (GERÇEK)

```
$ gcc -std=c17 -Wall -Wextra -pedantic -Wshadow -c H09_01_kapsam_golgeleme.c
H09_01_kapsam_golgeleme.c:21:9: warning: declaration of ‘x’ shadows a global declaration [-Wshadow]
H09_01_kapsam_golgeleme.c:24:13: warning: declaration of ‘x’ shadows a previous local [-Wshadow]
```

KAPSAMI BİTMİŞ AD: DERLEME HATASI (H09_K01, KASITLI HATA)

```
$ gcc -std=c17 -Wall -Wextra -pedantic -g H09_K01_kapsam_disi.c -o kapsam_disi
H09_K01_kapsam_disi.c:19:24: error: ‘i’ undeclared (first use in this function)
H09_K01_kapsam_disi.c:20:26: error: ‘ara’ undeclared (first use in this function)
```

Kural: değişkeni **en dar kapsamda** bildirin; `for (int i = …)` sayacını döngüde tutun; gölgelemeyi `-Wshadow` ile görünür kılın.

TAHMİN ET · KAPSAM · GÖLGELEME

Global `a = 5`, `main`'de `a = 1`, iç blokta `a = 2`. Program ne yazdırır?

quiz/H09_Q04_golgeleme.c · satır 7–19

```c
int a = 5;

int main(void)
{
    int a = 1;
    {
        int a = 2;
        a = a + 10;
    }
    a = a + 100;
    printf("%d\n", a);
    return 0;
}
```

A) 101

B) 112

C) 105

D) 110

Ömür: otomatik ve statik

H09_02_static_sayac.c · satır 10–17

```c
void ziyaret(void)
{
    int otomatik = 0;              /* her çağrıda yeniden oluşur ve 0 olur */
    static int statik = 0;         /* program başında bir kez 0 olur */
    otomatik++;
    statik++;                      /* değeri çağrılar arasında korunur */
    printf("%d. çağrı: otomatik = %d, statik = %d\n", statik, otomatik, statik);
}
```

GERÇEK ÇIKTI

```
$ ./static_sayac
1. çağrı: otomatik = 1, statik = 1
2. çağrı: otomatik = 1, statik = 2
3. çağrı: otomatik = 1, statik = 3
4. çağrı: otomatik = 1, statik = 4
```

Bildirim	Kapsam	Ömür	Bölge
fonksiyonda int x;	blok	otomatik	yığın
fonksiyonda static int x;	blok	statik	data/bss
dışarıda int x;	dosya (dış bağl.)	statik	data/bss
dışarıda static int x;	dosya (iç bağl.)	statik	data/bss

Modül = arayüz (.h) + gerçekleme (.c)

H09_04_cok_dosyali/matematik_util.h · satır 5–15

ARAYÜZ: bildirimler

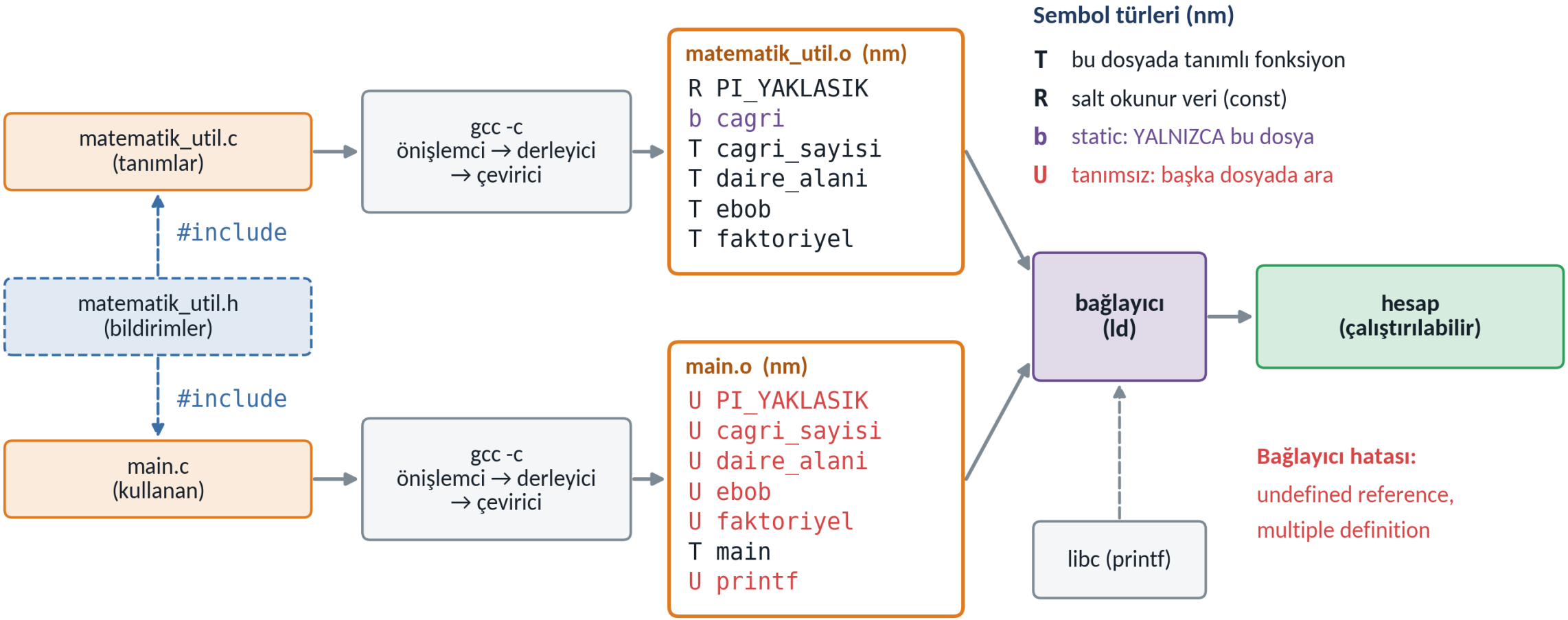
```
5  #ifndef MATEMATIK_UTIL_H          /* başlık koruyucusu (include guard) */
6  #define MATEMATIK_UTIL_H
7
8  extern const double PI_YAKLASIK;    /* bildirim: tanımı başka dosyada */
9
10 long long faktoriyel(int n);
11 int ebob(int a, int b);
12 double daire_alani(double r);
13 int cagri_sayisi(void);
14
15 #endif /* MATEMATIK_UTIL_H */
```

H09_04_cok_dosyali/matematik_util.c · satır 4–8

GERÇEKLEME: tanımlar

```
4  #include "matematik_util.h"        /* kendi başlığı: bildirim-tanım uyumu */
5
6  const double PI_YAKLASIK = 3.14159265358979;  /* extern'in tanımı */
7
8  static int cagri = 0;              /* dosya kapsamı + static: iç bağlantı */
```

Derleme hattı: önişlemci → derleyici → bağlayıcı



Derleme hatası (sözdizimi, bildirilmemiş ad): gcc -c adımında

Makro bir metin değiştirmedir

Kaynakta yazılan	Önişlemciden sonra (gcc -E)	Derleyicinin hesabı	Sonuç
`KARE_YANLIS(1 + 2)`	`1 + 2 * 1 + 2`	`1 + (2 * 1) + 2`	5
`KARE(1 + 2)`	`((1 + 2) * (1 + 2))`	`3 * 3`	9
`100 / KARE_YANLIS(5)`	`100 / 5 * 5`	`(100 / 5) * 5`	100
`100 / KARE(5)`	`100 / ((5) * (5))`	`100 / 25`	4
`kare(1 + 2)`	`kare(1 + 2)`	önce 1 + 2 = 3, sonra kare(3)	9

02

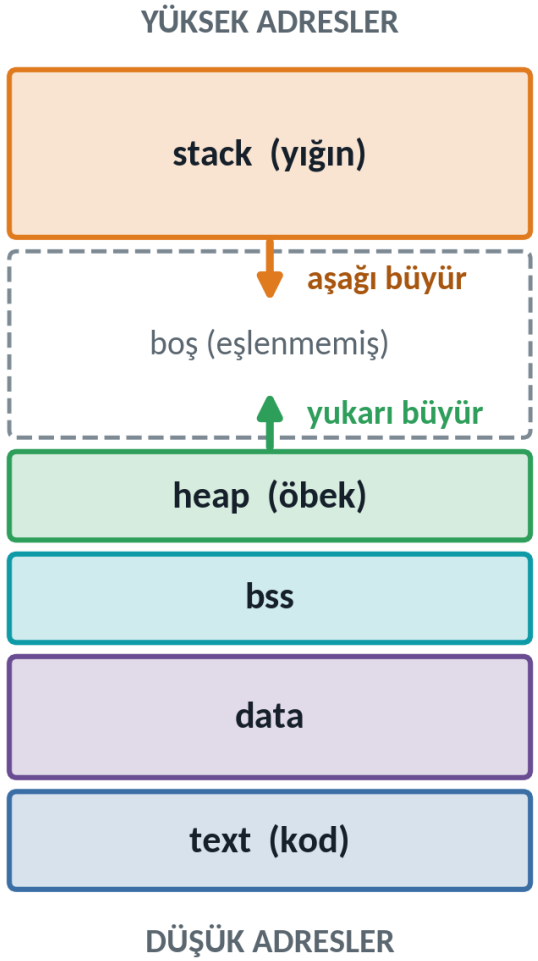
Çağrı yığını ve özyineleme

Her çağrı yığında bir çerçeve açar; bir fonksiyon kendini çağırdığında da

KAPUTUN ALTINDA

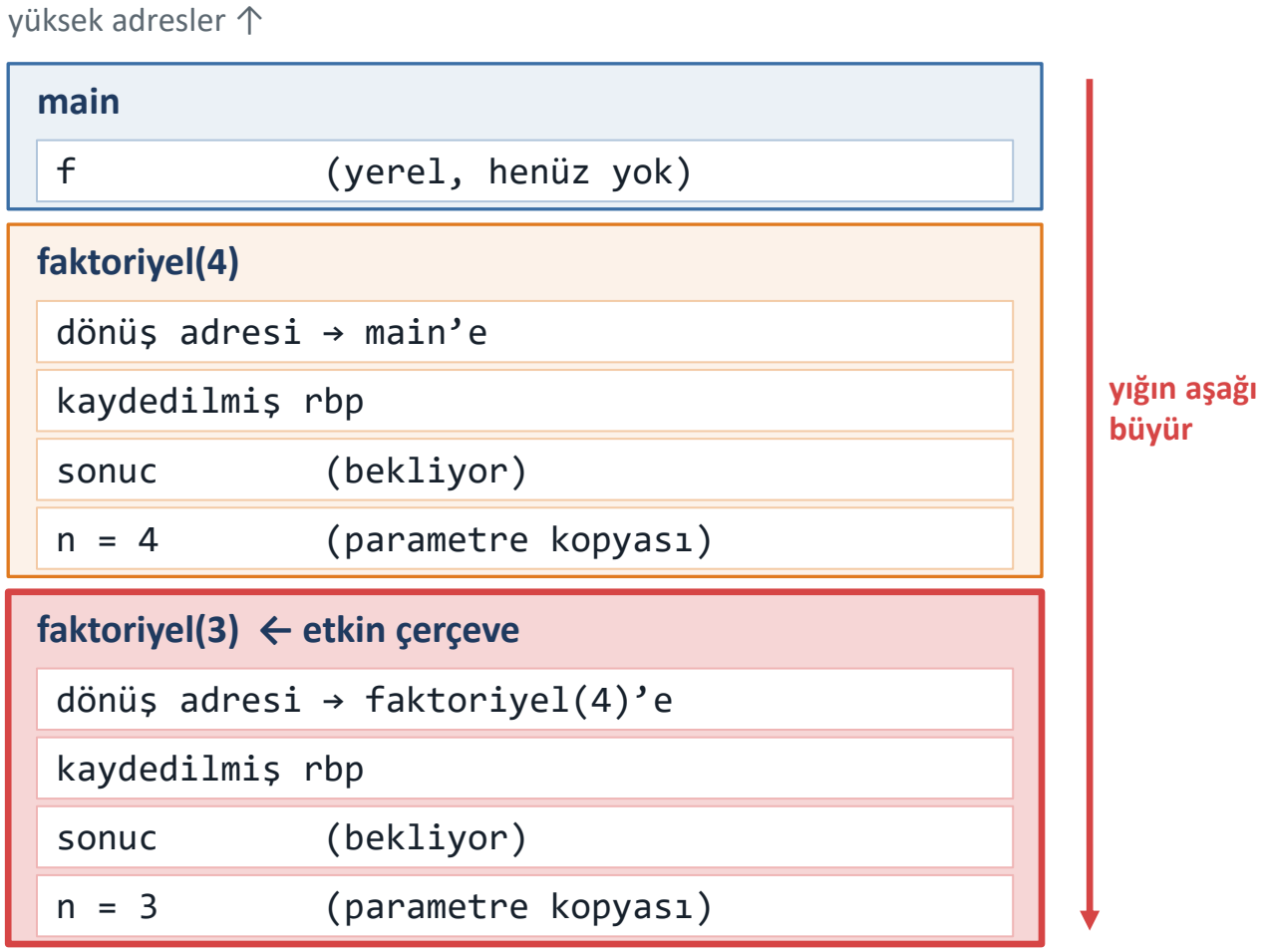
Bir sürecin bellek haritası

H09_K02 · gerçek adresler (setarch -R: ASLR kapalı)



`yerel`	`0x7fffffff8f1c`	stack (yığın)	yerel değişkenler, parametreler, dönüş adresleri: ÇERÇEVELER
`*obek`	`0x5555555592a0`	heap (öbek)	malloc ile alınan bellek (H12)
`ilk_degersiz`	`0x55555555801c`	bss	ilk değersiz global/static (0 yapılır)
`ilk_degerli`	`0x555555558010`	data	ilk değerli global/static
`statik_yerel`	`0x555555558014`		
`fonksiyon`	`0x5555555551a9`	text (kod)	makine kodu: fonksiyonlar

Yığın çerçevesi: call dönüş adresini iter, ret çeker



- `call faktoriyel`: dönüş adresini yığına **iter** (push) ve fonksiyona atlar
- Fonksiyon kendi **çerçevesini** açar: parametre kopyası, yereller, kaydedilmiş yazmaç
- `ret`: tepedeki dönüş adresini **çeker** (pop) ve oraya döner
- Son giren ilk çıkar (**LIFO**): dönüşler çağrıların tersi sırasıyla, karışmadan
- Ölçtük (K03): faktoriyel çerçevesi `-O0`'da **64 bayt**, `-O2`'de **32 bayt**

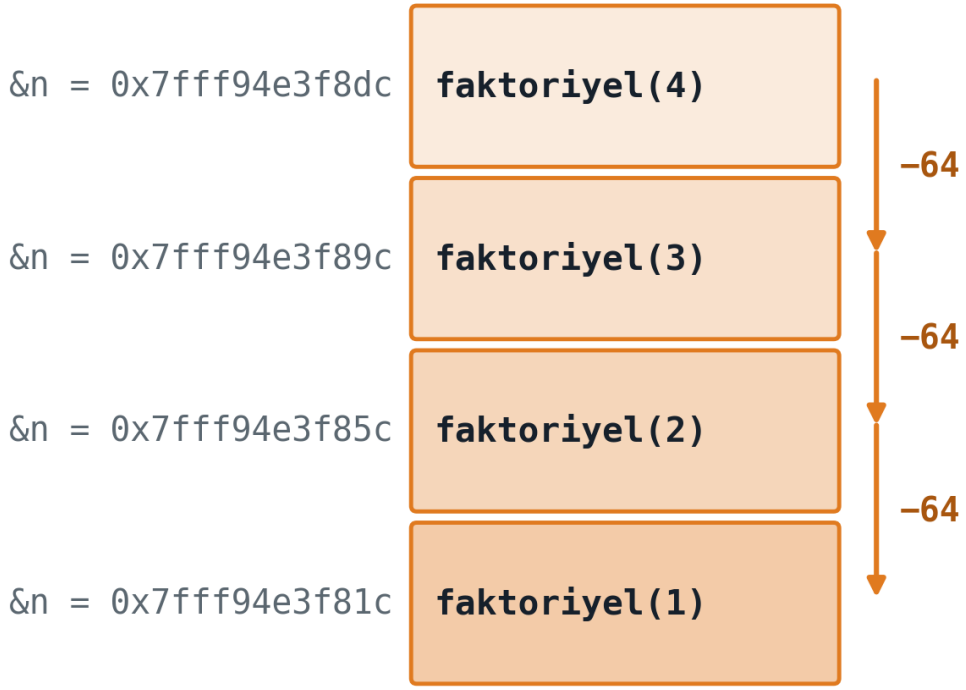
*Not: push/pop çizimlerinde en yeni çerçeveyi **üstte** çiziyoruz (tabak yığını); burada **adres** düzeni.*

KAPUTUN ALTINDA

Özyineleme derinleştikçe &n adresleri azalıyor

gcc -O0 (varsayılan)

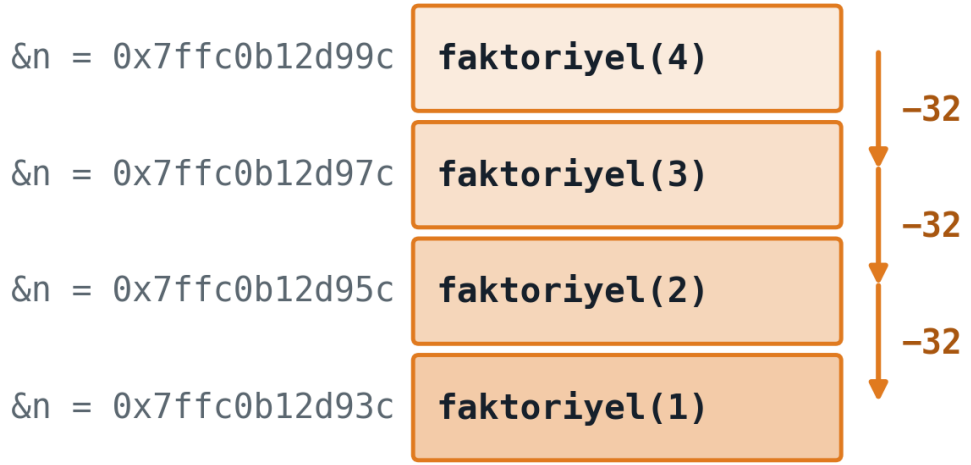
ardışık iki çerçeve arası: 64 bayt



adresler AZALIYOR: yığın aşağı büyür

gcc -O2 (iyileştirilmiş)

ardışık iki çerçeve arası: 32 bayt



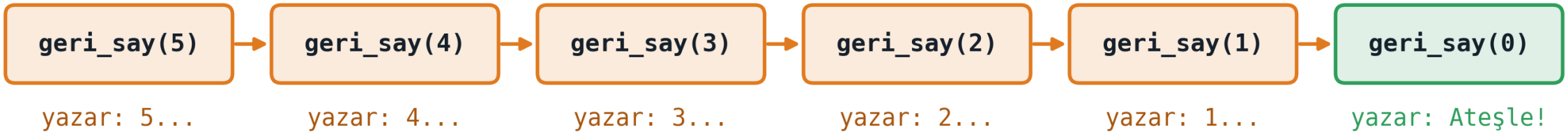
adresler AZALIYOR: yığın aşağı büyür

Özyinelemenin üç parçası

```
10  void geri_say(int n)
11  {
12      if (n == 0) {                   /* temel durum: artık çağrı yok */
13          printf("Ateşle!\n");
14          return;
15      }
16      printf("%d...\n", n);
17      geri_say(n - 1);                /* özyinelemeli adım: n küçülüyor */
18  }
```

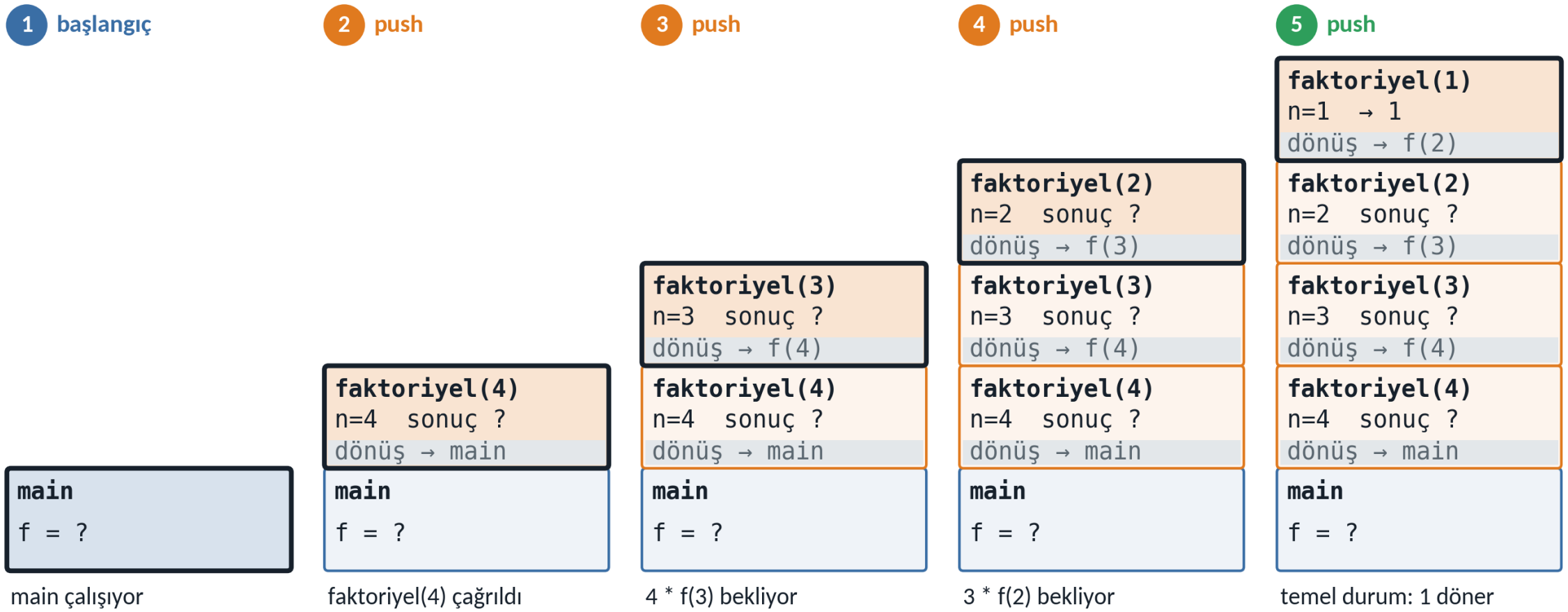
- temel durum: artık çağrı yok
- bu çerçevenin işi
- özyinelemeli adım

Her çağrı problemi bir küçültür: 5 → 4 → … → 0

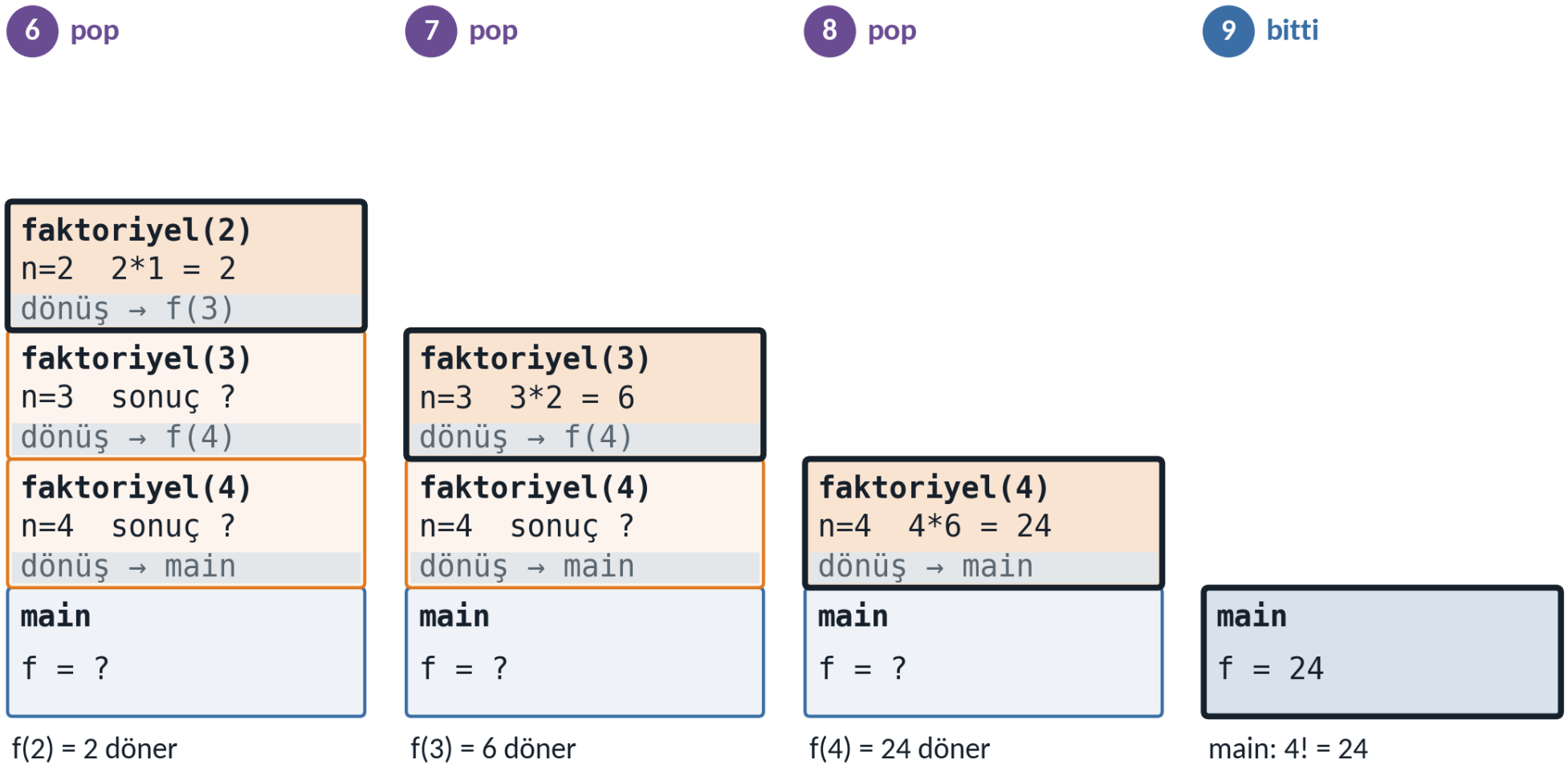


geri_say(0) temel durumdur; dönüşler bu noktadan başlar ve çerçeveler ters sırayla kapanır

faktoriyel(4): çerçeveler açılıyor (push)



faktoriyel(4): çerçeveler kapanıyor (pop)



TAHMİN ET · ÖZYİNELEME · DÖNÜŞTE İŞ

f(3) ne yazdırır?

quiz/H09_Q06_donuste_yaz.c · satır 7–14

```c
void f(int n)
{
    if (n == 0) {
        return;
    }
    f(n - 1);
    printf("%d ", n);
}
```

A 3 2 1

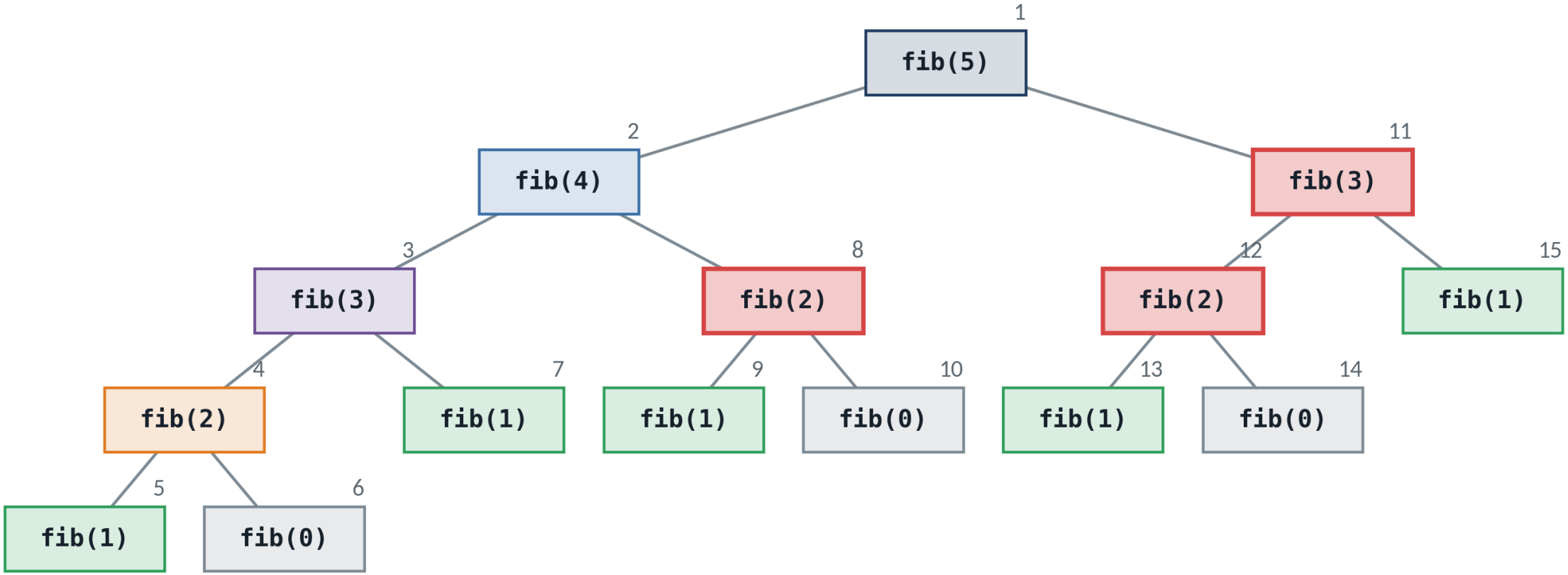
B 1 2 3

C 3

D 0 1 2 3

fib(5): özyineleme ağacı ve tekrarlanan hesap

Köşedeki sayı = çağrı sırası; kırmızı = daha önce hesaplanmış fib(3) ya da fib(2)'nin YENİDEN hesaplanması



Gerçek çıktı (H09_K09): Toplam 15 çağrı; fib(2) 3 kez hesaplandı. • fib(3): 2 kez, fib(1): 5 kez, fib(0): 3 kez

TAHMİN ET · ÖZYİNELEME AĞACI

`fib(5)` hesaplanırken `fib` toplam kaç kez çağrılır?

quiz/H09_Q07_fib_cagri.c · satır 7–16

```
 7  int cagri = 0;
 8
 9  int fib(int n)
10  {
11      cagri++;
12      if (n < 2) {
13          return n;
14      }
15      return fib(n - 1) + fib(n - 2);
16  }
```

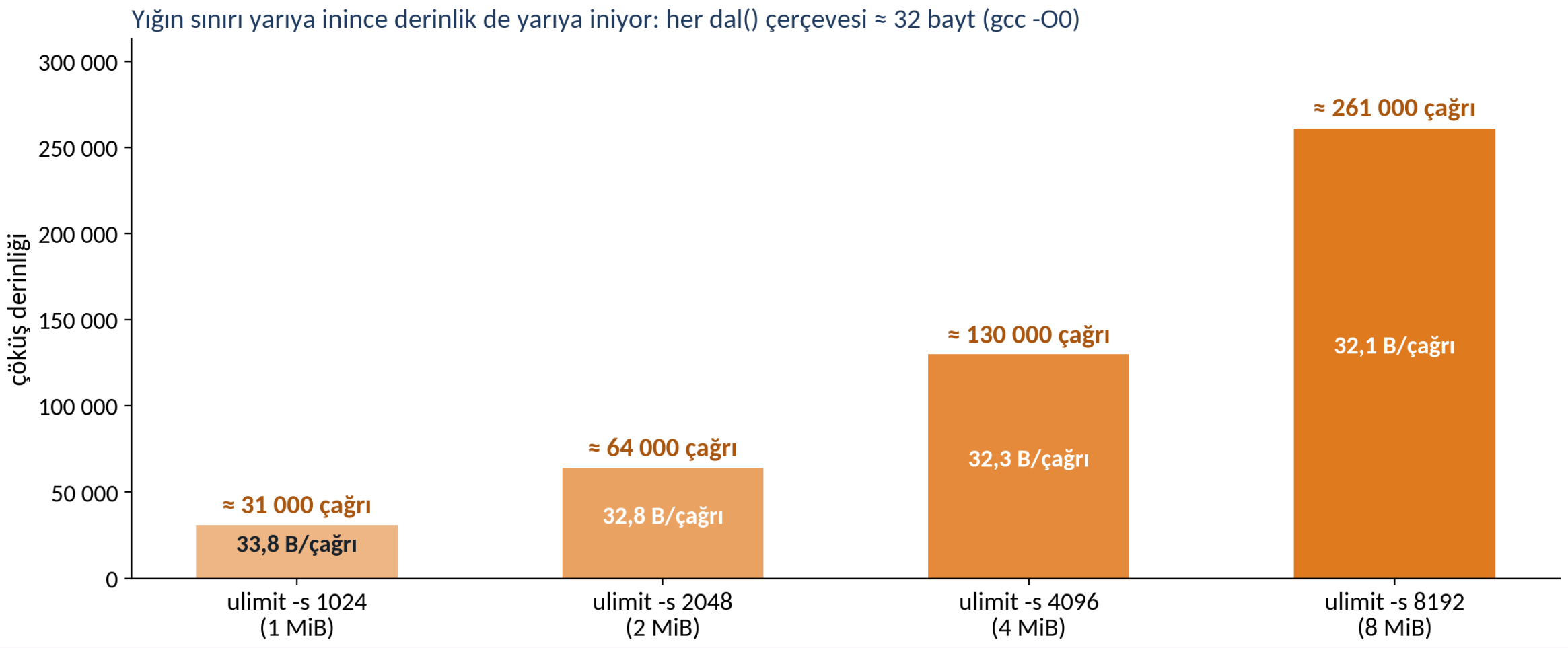
A 5

B 8

C 9

D 15

Yığın taşması: derinlik yığın boyuyla orantılı



Yinelemeli ↔ özyinelemeli: aynı problem, iki biçim

H09_K07_faktoriyel_saf.c · satır 11–18

```c
long long faktoriyel_dongu(int n)   /* yinelemeli: döngü + biriktirici */
{
    long long sonuc = 1;
    for (int i = 2; i <= n; i++) {
        sonuc *= i;
    }
    return sonuc;
}
```

H09_K07_faktoriyel_saf.c · satır 20–26

```c
long long faktoriyel(int n)         /* özyinelemeli: tanımın kendisi */
{
    if (n <= 1) {                   /* temel durum: 0! = 1! = 1 */
        return 1;
    }
    return n * faktoriyel(n - 1);   /* n! = n · (n-1)! */
}
```

Bitiş koşulu → temel durum · **biriktirici** → dönüş değerindeki işlem · **güncelleme** → argüman

03

Örnek merdiveni

Çok dosyalı modülden Hanoi Kulelerine: on üç örnek, gerçek çıktılarla

ÖRNEK 4 · ★★

Örnek 4: ayrı derle, bağla, çalıştır

H09_04_cok_dosyali/main.c · satır 12–16

```
12      printf("10! = %lld\n", faktoriyel(10));
13      printf("EBOB(84, 36) = %d\n", ebob(84, 36));
14      printf("Yarıçapı 2 olan dairenin alanı = %.4f\n", daire_alani(2.0));
15      printf("PI_YAKLASIK = %.5f (extern ile)\n", PI_YAKLASIK);
16      printf("Modül fonksiyonları %d kez çağrıldı.\n", cagri_sayisi());
```

AYRI DERLEME, BAĞLAMA, ÇALIŞTIRMA (GERÇEK)

```
$ gcc -std=c17 -Wall -Wextra -pedantic -g -c matematik_util.c
$ gcc -std=c17 -Wall -Wextra -pedantic -g -c main.c
$ gcc matematik_util.o main.o -o hesap
$ ./hesap
10! = 3628800
EBOB(84, 36) = 12
Yarıçapı 2 olan dairenin alanı = 12.5664
PI_YAKLASIK = 3.14159 (extern ile)
Modül fonksiyonları 3 kez çağrıldı.
```

ÖRNEK 6 · ★★ · AMİRAL ÖRNEK

Örnek 6: problem, girdi–işlem–çıktı ve sözde kod

Problem. n! değerini tanımın kendisiyle (n! = n · (n − 1)!) özyinelemeli hesaplayın ve her çağrının başlayıp bittiği anı ekranda izleyin.

GİRDİ

- n (0 ≤ n ≤ 20)
- 20!: `long long`'a sığan son faktöriyel

→

İŞLEM

- Temel durum: n ≤ 1 → 1
- Adım: n · faktoriyel(n − 1)
- Girişte ve dönüşte izleme satırı

→

ÇIKTI

- n!
- Çerçevelerin açılış/kapanış sırası

Sözde kod

```
faktoriyel(n):
    derinlik ← derinlik + 1              ▷ push: izleme satırı yaz
    n ≤ 1 ise sonuc ← 1, değilse sonuc ← n × faktoriyel(n − 1)
    dönüş satırını yaz; derinlik ← derinlik − 1; sonuc'u döndür    ▷ pop
```

Örnek 6: kod

H09_06_faktoriyel_yigin.c · satır 13–27

```
13  long long faktoriyel(int n)
14  {
15      derinlik++;                     /* PUSH: yeni çerçeve */
16      printf("%*s→ faktoriyel(%d) çağrıldı   [derinlik %d]\n",
17             2 * derinlik, "", n, derinlik);
18      long long sonuc;
19      if (n <= 1) {
20          sonuc = 1;                  /* temel durum */
21      } else {
22          sonuc = n * faktoriyel(n - 1);  /* özyinelemeli adım */
23      }
24      printf("%*s← faktoriyel(%d) = %lld döndü\n", 2 * derinlik, "", n, sonuc);
25      derinlik--;                     /* POP: çerçeve kapanır */
26      return sonuc;
27  }
```

- Satır 15 / 25: bütün çağrılar **aynı** `static` sayacı paylaşır; `%*s` girintisi derinliği gösterir
- Satır 22: çarpma ancak `faktoriyel(n - 1)` **döndükten sonra** yapılır; çerçeve bu yüzden bekler

Örnek 6: faktoriyel(4) izleme tablosu

Adım	Olay	Çağrı / dönüş	Çerçeve sayısı (olaydan sonra)	Bekleyen iş	Dönen değer
1	push	main → f(4)	2	4 * f(3)	—
2	push	f(4) → f(3)	3	3 * f(2)	—
3	push	f(3) → f(2)	4	2 * f(1)	—
4	**push**	**f(2) → f(1)**	**5**	**temel durum**	**—**
5	pop	f(1) → f(2)	4	—	1
6	pop	f(2) → f(3)	3	2 * 1	2
7	pop	f(3) → f(4)	2	3 * 2	6
8	**pop**	**f(4) → main**	**1**	**4 * 6**	**24**

Örnek 6: çalıştırma ve test

GERÇEK ÇIKTI

```
$ ./faktoriyel_yigin
  → faktoriyel(4) çağrıldı   [derinlik 1]
    → faktoriyel(3) çağrıldı   [derinlik 2]
      → faktoriyel(2) çağrıldı   [derinlik 3]
        → faktoriyel(1) çağrıldı   [derinlik 4]
        ← faktoriyel(1) = 1 döndü
      ← faktoriyel(2) = 2 döndü
    ← faktoriyel(3) = 6 döndü
  ← faktoriyel(4) = 24 döndü
4! = 24
```

TEST: DÖNGÜ = ÖZYİNELEME

```
$ ./faktoriyel_saf
 0! = 1 = 1
 5! = 120 = 120
10! = 3628800 = 3628800
15! = 1307674368000 = 1307674368000
20! = 2432902008176640000 = 2432902008176640000
```

- Sınır **0!**, normal **5!**, üst sınır **20!**: iki sürüm aynı
- **Değiştir:** `faktoriyel(21)`'i `-fsanitize=undefined` ile çalıştırın

ÖRNEK 7 · ★★

Örnek 7: 1..N toplamı ve üs alma

H09_07_toplam_us.c · satır 11–25

```c
long long toplam(int n)
{
    if (n == 0) {                   /* temel durum: boş toplam */
        return 0;
    }
    return n + toplam(n - 1);       /* n + (1..n-1 toplamı) */
}

long long us_al(long long taban, int us)
{
    if (us == 0) {                  /* temel durum: x^0 = 1 */
        return 1;
    }
    return taban * us_al(taban, us - 1);
}
```

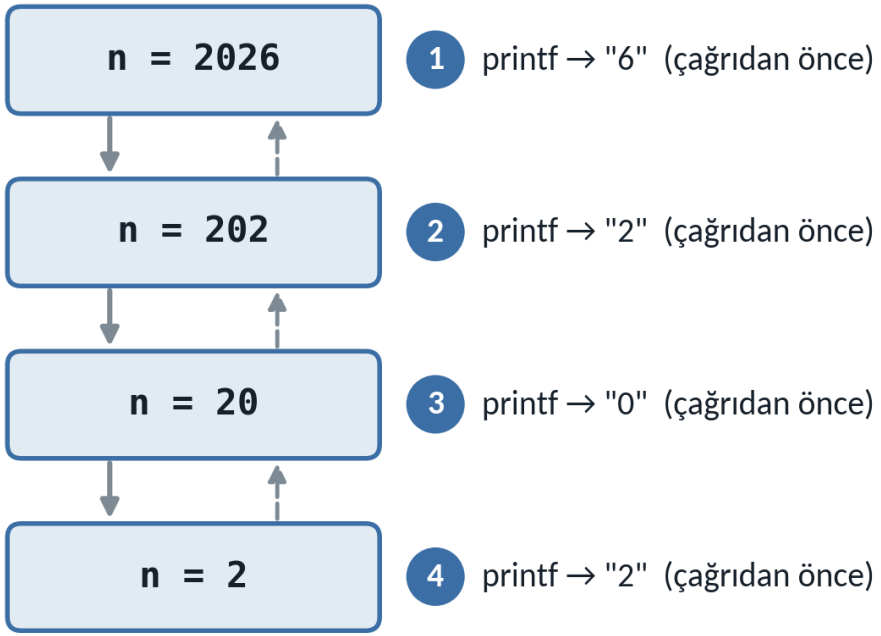
GERÇEK (SEÇİLİ)

```
$ ./toplam_us
toplam(4) = 10
us_al(2, 10) = 1024
us_al(3, 4)  = 81
us_al(-2, 5) = -32
us_al(7, 0)  = 1
```

- Temel durumlar: **boş toplam 0**, **$x^0 = 1$**
- `toplam(100)` = 5050: 101 çerçeve
- Formül varsa formül, yoksa döngü

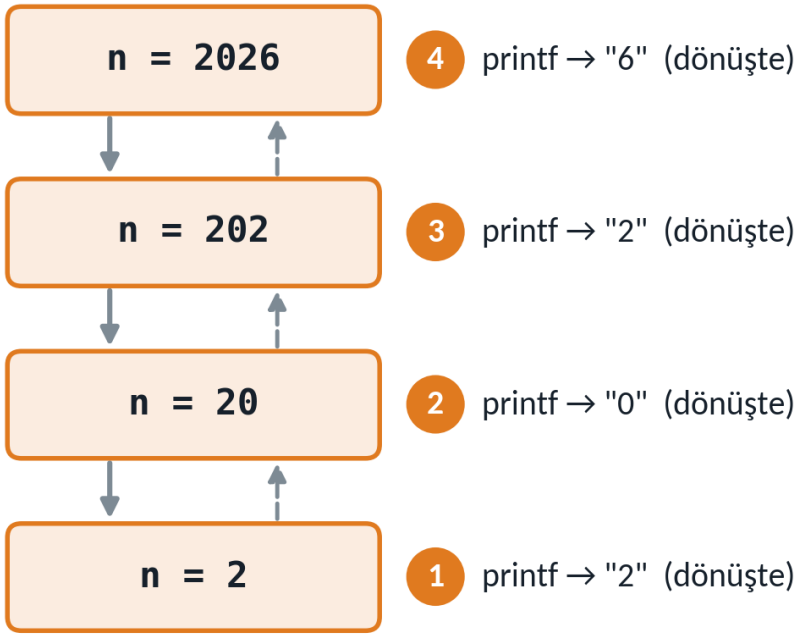
Örnek 8: iş önce mi, iş dönüşte mi?

ters_yaz(2026): İŞ ÖNCE (inerken yaz)




```
ekranda: 6202          → iniş  ⇢ dönüş
```

duz_yaz(2026): İŞ DÖNÜŞTE (çıkarken yaz)




```
ekranda: 2026          → iniş  ⇢ dönüş
```

ÖRNEK 9 · ★★ · AMİRAL ÖRNEK

Örnek 9: onluk sayıyı ikili yazdırma

H09_09_ikili_yazdir.c · satır 18–24

```c
18  void ikili_yaz(unsigned int n)      /* özyinelemeli çözüm */
19  {
20      if (n >= 2) {
21          ikili_yaz(n / 2);           /* önce üst bitler */
22      }
23      printf("%u", n % 2);            /* dönüşte bu çerçevenin biti */
24  }
```

Çerçeve	n	çağrı	dönüşte yazılan	sıra
1	13	ikili_yaz(6)	13 % 2 = 1	4.
2	6	ikili_yaz(3)	6 % 2 = 0	3.
3	3	ikili_yaz(1)	3 % 2 = 1	2.
4	1	—	1 % 2 = 1	1.

H07_10 yöntemi (gerçek): kalanlar **1011** ← ters sırada

GERÇEK (SEÇİLİ SATIRLAR)

```
13 = 1101 (ikili)
   0 = 0
   1 = 1
   4 = 100
2026 = 11111101010
```

ÖRNEK 10 · ★★

Örnek 10: özyinelemeli EBOB (Öklid)

H09_10_ebob_ozyinelemeli.c · satır 11–18

```c
int ebob(int a, int b)
{
    printf("  ebob(%d, %d)\n", a, b);  /* her çağrıyı göster */
    if (b == 0) {                      /* temel durum */
        return a;
    }
    return ebob(b, a % b);             /* özyinelemeli adım: b kesin küçülür */
}
```

EBOB(48, 18) (GERÇEK)

```
  ebob(48, 18)
  ebob(18, 12)
  ebob(12, 6)
  ebob(6, 0)
EBOB(48, 18) = 6
```

EBOB(18, 48) (GERÇEK)

```
  ebob(18, 48)
  ebob(48, 18)
  ebob(18, 12)
  ebob(12, 6)
  ebob(6, 0)
EBOB(18, 48) = 6
```

Örnek 11: temel durum hiç sağlanmazsa

H09_11_yigin_tasmasi.c · satır 17–27

```c
long dal(long derinlik, long hedef)
{
    if (derinlik == hedef) {        /* temel durum: hedef derinlik */
        return derinlik;
    }
    if (derinlik % ARALIK == 0) {
        printf("derinlik %ld\n", derinlik);
        fflush(stdout);             /* çökmeden önce çıktı kaybolmasın */
    }
    return dal(derinlik + 1, hedef);
}
```

GERÇEK ÇIKTI

```
$ ./yigin_tasmasi; echo "çıkış durumu: $?"
Hedef derinlik (-1: hiç ulaşılamaz): -1
derinlik 10000
…
derinlik 260000
bash: line 1:  8849 Segmentation fault      ./yigin_tasmasi
çıkış durumu: 139
```

ulimit -s	çöküş derinliği
8 192 KiB	≈ 261 000
4 096 KiB	≈ 130 000
2 048 KiB	≈ 64 000
1 024 KiB	≈ 31 000

ÖRNEK 12 · ★★★

Örnek 12: Fibonacci’de çağrı sayısı üstel büyür

H09_12_fibonacci_cagri.c · satır 12–21

```c
static long long cagri = 0;          /* fib kaç kez çağrıldı? */

long long fib(int n)                 /* özyinelemeli: tanımın birebir kopyası */
{
    cagri++;
    if (n < 2) {                     /* iki temel durum: fib(0) = 0, fib(1) = 1 */
        return n;
    }
    return fib(n - 1) + fib(n - 2);
}
```

GERÇEK ÖLÇÜM (SÜRELER MAKİNEYE GÖRE DEĞİŞİR, ÇAĞRI SAYILARI DEĞİŞMEZ)

```
$ ./fibonacci_cagri
  n     fib(n)   çağrı sayısı  özyinelemeli       döngü  sonuç
  5          5             15        0.0 ms    0.000 ms  aynı
 10         55            177        0.0 ms    0.001 ms  aynı
 20       6765          21891        0.1 ms    0.000 ms  aynı
 30     832040        2692537       12.2 ms    0.001 ms  aynı
 35    9227465       29860703       89.2 ms    0.001 ms  aynı
 40  102334155      331160281     1032.5 ms    0.000 ms  aynı
```

ÖRNEK 13 · ★★★

Örnek 13: Hanoi Kuleleri

H09_13_hanoi.c · satır 12–23

```c
void hanoi(int n, char kaynak, char hedef, char yardimci, int yaz)
{
    if (n == 0) {                    /* temel durum: taşınacak disk yok */
        return;
    }
    hanoi(n - 1, kaynak, yardimci, hedef, yaz);   /* 1) üstteki n-1 disk */
    hamle++;                                      /* 2) en büyük disk */
    if (yaz) {
        printf("%2lld. hamle: disk %d  %c -> %c\n", hamle, n, kaynak, hedef);
    }
    hanoi(n - 1, yardimci, hedef, kaynak, yaz);   /* 3) n-1 disk üstüne */
}
```

GERÇEK (SEÇİLİ SATIRLAR)

n	sayılan hamle	2^n - 1
3	7	7
10	1023	1023
15	32767	32767
20	1048575	1048575

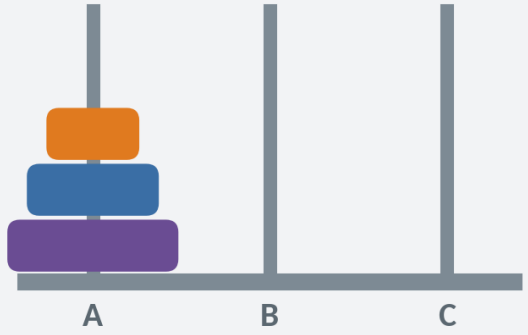
H(n) = H(n – 1) + 1 + H(n – 1)

Bir disk eklemek hamleyi **iki katına çıkarıp bir ekler**: 1, 3, 7, 15, 31, … = **$2^n - 1$**

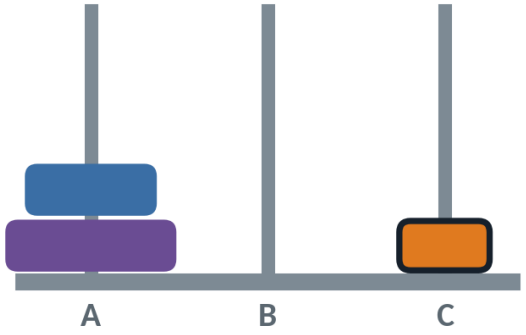
64 disk: 18 446 744 073 709 551 615 = ULLONG_MAX

3 disk, 7 hamle (gerçek hamle listesinden)

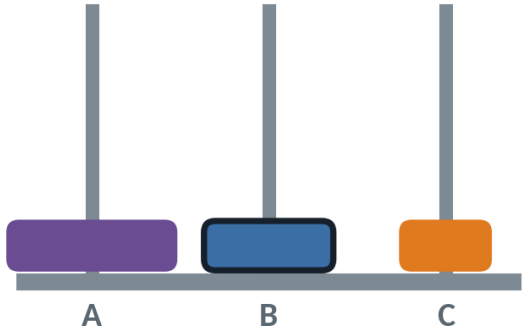
başlangıç: A'da 3 disk



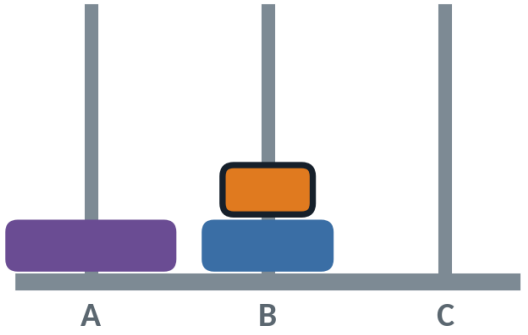
1. hamle: disk 1 A → C



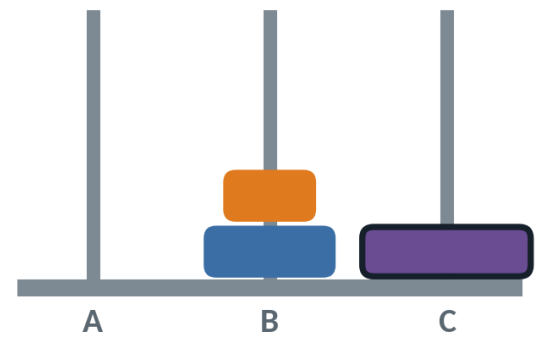
2. hamle: disk 2 A → B



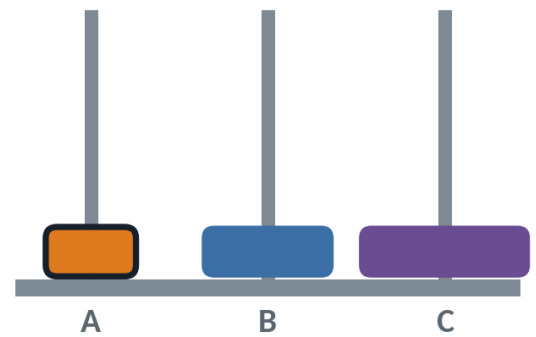
3. hamle: disk 1 C → B



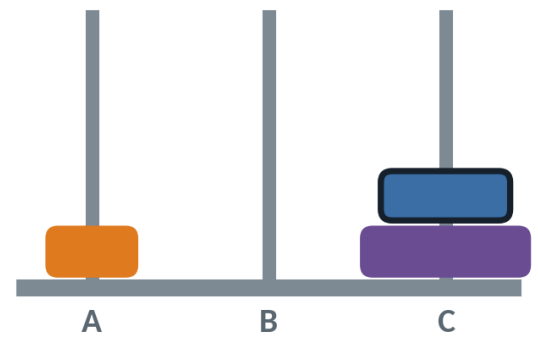
4. hamle: disk 3 A → C



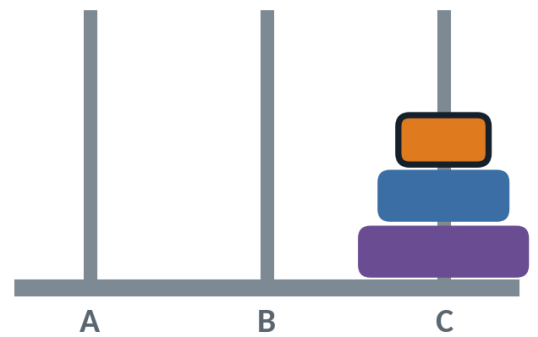
5. hamle: disk 1 B → A



6. hamle: disk 2 B → C



7. hamle: disk 1 A → C



HATA AVCISI

HA1–HA2: temel durum yok ya da hiç ulaşılamıyor

hata_avcisi/H09_HA2_atlanan_temel_durum.c · satır 11–17

```c
int azalan_toplam(int n)
{
    if (n == 0) {                    /* yalnızca çift n bu duruma iner */
        return 0;
    }
    return n + azalan_toplam(n - 2);
}
```

GERÇEK ÇIKTI

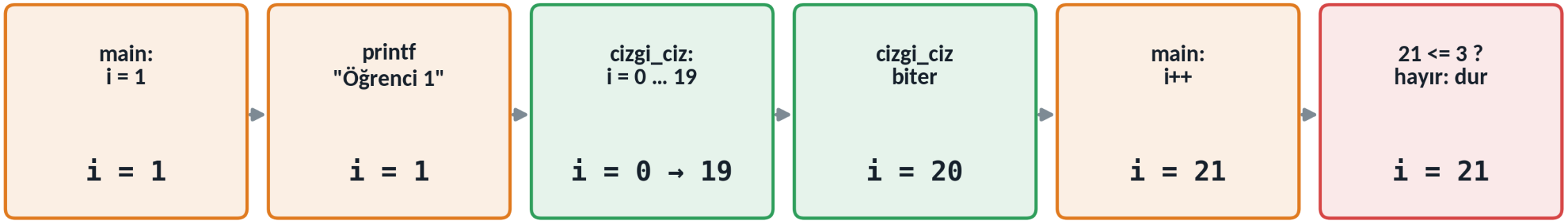
```
$ ./ha2; echo "çıkış durumu: $?"
azalan_toplam(10) = 30
bash: line 1:  9440 Segmentation fault      ./ha2
çıkış durumu: 139
```

- **HA1:** temel durum hiç yok → `-Winfinite-recursion` uyarısı, çökme (139)
- **HA2:** 7 → 5 → 3 → 1 → −1: 0'ın **üzerinden atlar**
- **Düzeltme:** aralık sına: `if (n <= 0)`

HATA AVCISI

HA3: global sayaç döngüyü bozuyor

TEK bir global i: iki fonksiyon aynı kutuyu değiştiriyor (H09_HA3)



GERÇEK ÇIKTI

```
Öğrenci 1

--------------------
```

Beklenen: 3 öğrenci satırı

Gerçek: döngü 1 turda bitti

Çözüm: her döngüye kendi yerel i'si

HATA AVCISI

HA4: başlık dosyasına tanım → multiple definition

hata_avcisi/H09_HA4_cok_tanim/sayac.h · satır 5–12

```
 5  #ifndef SAYAC_H
 6  #define SAYAC_H
 7
 8  int toplam_cagri = 0;                /* HATA: tanım başlıkta! */
 9
10  void say(void);
11
12  #endif /* SAYAC_H */
```

İKİ DOSYA DA DERLENİR, BAĞLAMA BAŞARISIZ (GERÇEK)

```
$ gcc -std=c17 -Wall -Wextra -pedantic -g -c sayac.c
$ gcc -std=c17 -Wall -Wextra -pedantic -g -c main.c
$ gcc sayac.o main.o -o sayac
/usr/bin/ld: main.o:sayac.h:8: multiple definition of `toplam_cagri'; sayac.o:sayac.h:8: first defined here
collect2: error: ld returned 1 exit status
```

Düzeltme: başlıkta `extern int toplam_cagri;` (bildirim), tanım yalnızca `sayac.c`'de. Koruyucu işe yaramaz: yalnızca **aynı** çeviri birimindeki kopyayı engeller.

HATA AVCISI

HA5–HA7: makro tuzakları ve kaybolan dönüş değeri

Hata	Belirti (gerçek)	Neden	Düzeltme
HA5 · #define FARK(a, b) a - b	FARK(10, 3 - 1) = 6; beklenen 8 · uyarı yok	10 - 3 - 1: öncelik	((a) - (b))
HA6 · KARE(i++)	GCC: operation on ‘i’ may be undefined [-Wsequence-point]	i iki kez değişir: tanımsız davranış	ayrı deyim ya da fonksiyon
HA7 · return unutuldu	GCC: toplam 2, ebob 6 · Clang: 0 ve 32703 (beklenen 10, 6)	dönüşsüz fonksiyonun değeri: tanımsız	return ebob(b, a % b);

HA7 · GCC (GERÇEK)

```
$ ./ha7          # gcc
basamak_toplami(2026) = 2  (beklenen 10)
ebob(48, 18) = 6  (beklenen 6)
```

HA7 · CLANG (GERÇEK)

```
$ ./ha7_clang    # aynı kaynak, clang
basamak_toplami(2026) = 0  (beklenen 10)
ebob(48, 18) = 32703  (beklenen 6)
```

“Bende çalışıyor” kanıt değildir: HA7’de GCC’li program EBOB için **şans eseri** doğru sonuç verdi; uyarılar (`-Wreturn-type`) kanıttır.

Mühendis gözüyle: kapsamı daralt, yığını hesapla

En dar kapsam

- Global yerine parametre ve dönüş değeri
- Paylaşılan durum: dosyada `static` + erişim fonksiyonu
- `-Wshadow` açık

Arayüz bir sözleşme

- Başlıkta yalnızca bildirim + koruyucu
- Gerçekleme değişse de arayüz aynı kalırsa yalnızca yeniden bağlama

Yığın bütçesi

- derinlik × çerçeve < yığın boyu
- 8 MiB: yüz binlerce; 1 MB (MSVC): on binlerce derinlik
- Gömülü sistemde yığın birkaç KB olabilir

Önce ölç, sonra karar ver

- Fibonacci: zarif ama üstel
- Hanoi: hem zarif hem gerekli ($2^n - 1$)
- Makro yerine fonksiyon ve `const`

MERAK KUTUSU

Kuyruk özyinelemesi: derleyici döngüye çevirebilir mi?

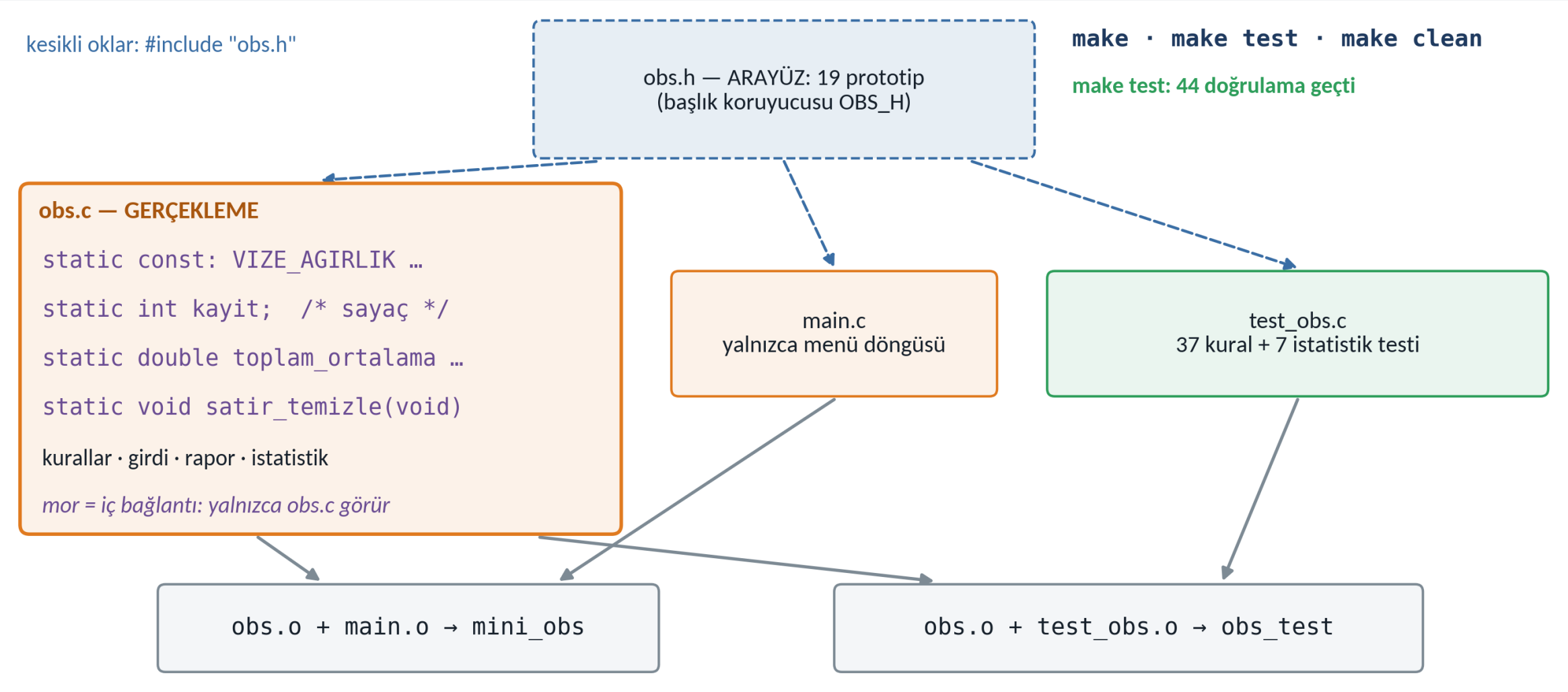
gcc -O0: kendini çağırır (call)

```
faktoriyel_kuyruk:
    endbr64
    push rbp
    mov rbp, rsp
    sub rsp, 16
    mov DWORD PTR [rbp-4], edi
    mov QWORD PTR [rbp-16], rsi
    cmp DWORD PTR [rbp-4], 1
    jg .L5
    mov rax, QWORD PTR [rbp-16]
    jmp .L6
.L5:
    mov eax, DWORD PTR [rbp-4]
    cdqe
    imul rax, QWORD PTR [rbp-16]
    mov edx, DWORD PTR [rbp-4]
    sub edx, 1
    mov rsi, rax
    mov edi, edx
    call faktoriyel_kuyruk
.L6:
    leave
    ret
```

gcc -O2: call YOK, döngü (jne)

```
faktoriyel_kuyruk:
    endbr64
    mov rax, rsi
    cmp edi, 1
    jle .L7
    movsx rcx, edi
    sub edi, 2
    lea rdx, [rcx-1]
    mov rsi, rdx
    sub rsi, rdi
    jmp .L10
.L11:
    sub rdx, 1
.L10:
    imul rax, rcx
    mov rcx, rdx
    cmp rdx, rsi
    jne .L11
.L7:
    ret
```

Mini-OBS v0.9: tek dosyadan modüllere



MİNİ-OBS PROJESİ

Mini-OBS v0.9: make ve kabul ölçütleri

MAKE (GERÇEK OTURUM)

```
$ make      # hiçbir şey değişmedi
make: 'mini_obs' is up to date.
$ touch obs.c && make
gcc -std=c17 -Wall -Wextra -pedantic -g -c obs.c
gcc -std=c17 -Wall -Wextra -pedantic -g obs.o main.o -o mini_obs
$ make test
gcc -std=c17 -Wall -Wextra -pedantic -g -c test_obs.c
gcc -std=c17 -Wall -Wextra -pedantic -g obs.o test_obs.o -o obs_test
./obs_test
Tüm testler geçti (44 doğrulama: 37 kural + 7 istatistik).
```

- Sıfır uyarı; `make`, `make test` çalışır
- **44** doğrulama geçer (37 kural + 7 istatistik)
- Kurallar (RTEÜ tablosu, baraj, şartlı DC) değişmedi

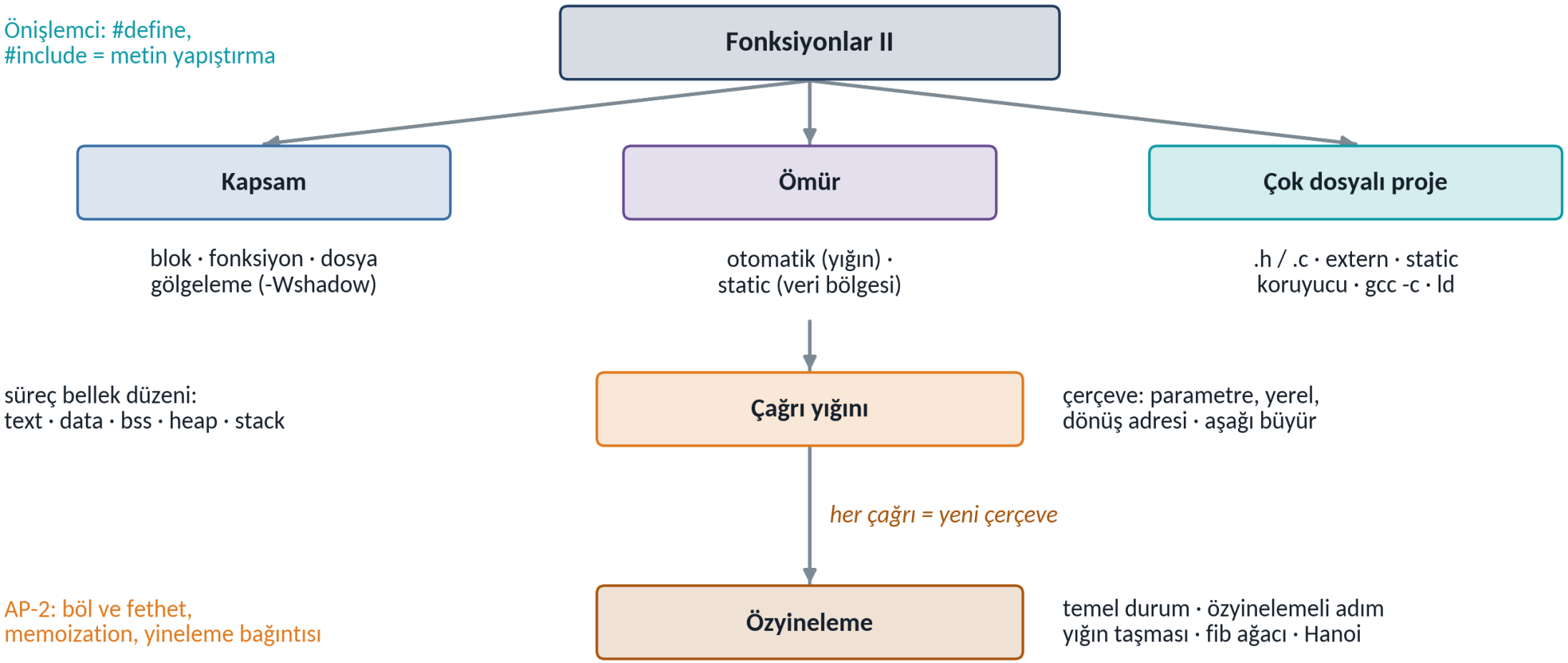
- v0.8 ile çıktılar **birebir aynı** (5 test; yalnızca sürüm satırı farklı)
- `main.c` istatistik değişkenlerine doğrudan erişmez
- Windows/MinGW: `mingw32-make`

AP-2 KÖPRÜSÜ

Fibonacci’nin bedeli ve böl ve fethet

- **Memoization / dinamik programlama:** her fib(k)’yı ilk hesaplandığında **not et**, sonra notu oku (AP-2, diziler H10’dan sonra)
- **Yineleme bağıntıları:** C(n) = C(n − 1) + C(n − 2) + 1 → çağrı sayısı **2 · fib(n + 1) − 1**; Hanoi: H(n) = 2H(n − 1) + 1
- **Böl ve fethet:** Hanoi’nin düşüncesi; AP-2’de **birleştirme sıralaması**, **hızlı sıralama**
- **Özyinelemeli ikili arama:** temel durum + yarıya inen problem

Özet: haftanın kavram haritası



Büyük Sorunun yanıtı

Kendini çağırınca ne olur?

Sıradan bir çağrıda olan: **yeni bir çerçeve** açılır, parametre ve yereller **ayrıdır**. faktoriyel(4)'te dört ayrı `n`, 64 bayt arayla.

Nereye döneceğini nasıl hatırlar?

Her `call` dönüş adresini yığına **iter**, her `ret` tepedekini **çeker**. LIFO: dönüşler çağrıların tersi sırasıyla.

Bedeli ne?

Temel durumsuz özyineleme ≈ **261 000** çağrıda 8 MiB'ı tüketti; fib(40): **331 160 281** çağrı; Hanoi 64 disk: evren yaşının **42,4** katı.

ÇIKIŞ BİLETİ

Dersten çıkmadan önce üç soru

1. Quiz S6'daki `f`'de `f(n - 1);` ile `printf` satırlarının yeri değişirse `f(3)` ne yazdırır? Neden?

2. Fonksiyon içindeki `static int sayac = 0;` ile dosya kapsamındaki `int sayac = 0;` arasındaki fark: kapsam mı, ömür mü?

3. Başlık dosyasına `int toplam = 0;` yazılırsa hata hangi aşamada çıkar ve neden? Nasıl düzeltilir?

KAPANIŞ

Haftaya hazırlık ve kaynaklar

Okuma

- King (2008): Bölüm 9.6 (özyineleme), 10 (program düzeni), 15 (büyük programlar)
- Kernighan ve Ritchie (1988): Bölüm 4

Web crash course

- Hafta 9 sayfası: yığın animasyonu, fib ağacı, oynanabilir Hanoi
- 10 soruluk quiz

Ödev ve atölye

- Alıştırmalar 9.1–9.12 (`kodlar/alistirmalar/`)
- Mini-OBS v0.8 → v0.9: `diff` ile doğrula

Gelecek hafta: Diziler I

- Bitişik bellek, `int notlar[100];`
- H07_10'daki bitleri bir diziye biriktirip tersten yazdıracağız
- Hazırlık: dizi hangi bellek bölgesinde olur?